

**INHA UNIVERSITY TASHKENT
DEPARTMENT OF CSE & ICE**

FALL SEMESTER 2024

SOC 3010 - OPERATING SYSTEMS

HOME ASSIGNMENT 3

INSTRUCTIONS :

- **ALL HOME ASSIGNMENTS ARE TO BE COMPLETED IN GROUPS OF 4 TO 6 MEMBERS**
- **SCREEN SHOTS ARE TO BE PROVIDED FOR ALL THE QUESTIONS**
- **HOME ASSIGNMENT REPORT SHOULD BE PREPARED USING THE TEMPLATE PROVIDED**
- **A HARDCOPY OF THE REPORT MUST BE HANDED IN BY THE TEAM LEADER BY VISITING MY OFFICE ON MONDAY 11TH NOVEMBER 2024 AFTERNOON ANYTIME BETWEEN 1.30PM & 4.30PM.**
-
- **EVERY MEMBER OF THE TEAM MUST UPLOAD THE SOFTCOPY OF THE REPORT AT THE IUT ECLASS PORTAL**
- **LAST DATE FOR SUBMISSION OF THE HOME ASSIGNMENT3 AT THE ECLASS PORTAL IS SUNDAY 10TH NOVEMBER 2024.**
- **LATE SUBMISSIONS ARE NOT ENTERTAINED, ADHERE TO THE DEADLINE STRICTLY**
- **READ THE QUESTIONS CORRECTLY & CAREFULLY**

QUESTIONS :

PART I : WRITING LINUX KERNEL MODULE

ALL MEMBERS INCLUDING THE TEAM LEADER MUST EXECUTE PART I QUESTIONS 2, 3, 4, 5 & 6 ON THEIR LAPTOPS/DESKTOPS AND PROVIDE THE SCREENSHOTS OF THE PROGRAMS & COMMANDS ENTERED ON THEIR SYSTEM, AND RESULTS OBTAINED ON THEIR SYSTEM.

HARDCOPY OF THE TEAM LEADER MUST CONTAIN ALL THESE SCREENSHOTS CLEARLY INDICATING THE NAME, ID OF THE MEMBERS INCLUDING THEIR SIGNATURE ON EVERY PAGE OF THE SCREEN SHOT SUBMITTED.

YOU ARE REQUIRED TO CLEARLY INDICATE THE CORRECT ANSWERS FOR ALL PARTS OF THE QUESTIONS a), b), c) or i) , ii), iii), iv), v), vi) IN ORDER TO GET FULL MARKS. ANY MISSING PART ANSWER WILL RESULT IN 0 MARKS

ANSWER FOR QUESTION 1 SHOULD BE TYPED BY THE INDIVIDUAL MEMBER IN HIS/HER REPORT

1. a) Describe all the steps required for developing Linux Kernel Modules.
b) List out all the Linux utilities to manipulate Kernel modules and provide suitable examples to explain their functions.
c) Provide the Makefile template that can be used to compile the kernel module.
2. Write a Kernel module to
 - a) Display THE ENTIRE 16 WEEK OPERATING SYTEM COURSE SYLLABUS GIVEN IN THE WEEK1 SYLLABUS SLIDES AT THE ECLASS PORTAL when module `__init` is invoked.
 - b) Display THE ENTIRE TERM PROJECT GUIDELINES GIVEN IN THE WEEK1 SYLLABUS SLIDES AT THE ECLASS PORTAL when module `__exit` is invoked.
 - c) **PROVIDE SCREEN SHOTS OF ALL THE RESULTS INCLUDING THE PROGRAM.**
3. Write a Kernel module to
 - a) Display your TEAM Information (specifying for each member of the TEAM, the following details using a suitable format - Student Full Name, Student ID, Level (Junior), Group No., Team No., Home Address, Ph. No., Email address, CGPA Grade upto Somophore level of every member, TERM PROJECT TITLE) when module `__init` is invoked.
 - b) Display your future plan (what each member wants to become after graduation) and goal in life when module `__exit` is invoked.
 - c) **PROVIDE SCREEN SHOTS OF ALL THE RESULTS INCLUDING THE PROGRAM.**
4. Write a Kernel module to
 - (i) pass parameters from Command line to input each TEAM member information,

- (ii) store (append) it in a file named `OSHA3TEAM_teamnumber` (example, `OSHA3TEAM_36`), file name must be passed from the command line and
- (iii) display the file contents. The parameters to be passed from the command line are Student Full Name, Student ID, Level (Junior), Group No., Team No., Home Address, Ph. No., Email address, CGPA Grade upto Somophore level of a member, TERM PROJECT TITLE, TERM PROJECT TEAM NO., filename. The default values to be stored initially would be the TEAM leader's information & filename.
- (iv) With every invocation of the module `__init` (i.e., when `insmod` is executed), a new TEAM member information containing Student Full Name, student ID, Level (Junior), Group No., Team No., Home Address, Ph. No., Email address, CGPA Grade upto Somophore level of that member, TERM PROJECT TITLE, TERM PROJECT TEAM NO., filename must be taken from the command line arguments and appended to the file.

FOR EXAMPLE: IF YOU HAVE SIX MEMBERS IN A TEAM, THEN YOU REQUIRE SIX `insmod` COMMANDS TO ADD INFORMATION ABOUT EACH MEMBER ONE AT A TIME WITH EACH EXECUTION OF `insmod` COMMAND, THAT IS, EACH INVOCATION OF THE MODULE `__init`

WITH INVOCATION OF EVERY COMMAND `insmod`, i.e., `__init` function INVOCATION, YOU ARE REQUIRED TO OPEN THE FILE IN WRITE APPEND MODE AND ADD EACH MEMBER INFORMATION ONE AT TIME FROM THE COMMAND LINE IN THE FOLLOWING FASHION :

IN THE COMMAND LINE OF FIRST `insmod` command, YOU SPECIFY THE FIRST MEMBER INFORMATION AS COMMAND LINE ARGUMENTS AND ADD THAT INFORMATION TO THE FILE OPENED WHEN YOU RUN `__init` FUNCTION.

IN THE COMMAND LINE OF SECOND `insmod` command, YOU SPECIFY THE SECOND MEMBER INFORMATION AS COMMAND LINE ARGUMENTS AND ADD THAT INFORMATION TO THE END OF THE FILE OPENED WHEN YOU RUN `__init` FUNCTION.

IN THE COMMAND LINE OF THIRD `insmod` command, YOU SPECIFY THE THIRD MEMBER INFORMATION AS

COMMAND LINE ARGUMENTS AND ADD THAT INFORMATION TO THE END OF THE FILE OPENED WHEN YOU RUN `__init` FUNCTION.

IN THE COMMAND LINE OF FOURTH `insmod` command, YOU SPECIFY THE FOURTH MEMBER INFORMATION AS COMMAND LINE ARGUMENTS AND ADD THAT INFORMATION TO THE END OF THE FILE OPENED WHEN YOU RUN `__init` FUNCTION.

IN THE COMMAND LINE OF FIFTH `insmod` command, YOU SPECIFY THE FIFTH MEMBER INFORMATION AS COMMAND LINE ARGUMENTS AND ADD THAT INFORMATION TO THE END OF THE FILE OPENED WHEN YOU RUN `__init` FUNCTION.

IN THE COMMAND LINE OF SIXTH `insmod` command, YOU SPECIFY THE SIXTH MEMBER INFORMATION AS COMMAND LINE ARGUMENTS AND ADD THAT INFORMATION TO THE END OF THE FILE OPENED WHEN YOU RUN `__init` FUNCTION.

- (v) With every invocation of `module __exit` (i.e., when `rmmod` is executed) Display the current contents of the file (file name `OSHA3TEAM_teamnumber`).**

FOR EXAMPLE : IF YOU HAVE SIX MEMBERS IN A TEAM, THEN YOU REQUIRE SIX `rmmod` COMMANDS TO REMOVE THE KERNEL MODULE SIX TIMES AS YOU HAVE ADDED THE KERNEL MODULE SIX TIMES USING `insmod` COMMAND

WITH INVOCATION OF EVERY COMMAND `rmmod`, i.e., `__exit` function INVOCATION, YOU ARE REQUIRED TO OPEN THE FILE IN READ MODE AND DISPLAY THE CONTENTS OF THE FILE IN THE FOLLOWING FASHION :

FIRST `rmmod` command EXECUTION DISPLAYS THE FIRST MEMBER INFORMATION STORED IN THE FILE.

SECOND `rmmod` command EXECUTION DISPLAYS THE LAST TWO MEMBERS INFORMATION STORED IN THE FILE.

THIRD `rmmod` command EXECUTION DISPLAYS THE LAST THREE MEMBERS INFORMATION STORED IN THE FILE.

FOURTH rmmmod command EXECUTION DISPLAYS THE LAST FOUR MEMBERS INFORMATION STORED IN THE FILE.

FIFTH rmmmod COMMAND EXECUTION DISPLAYS THE LAST FIVE MEMBERS INFORMATION STORED IN THE FILE.

SIXTH rmmmod COMMAND EXECUTION DISPLAYS THE LAST SIX MEMBERS INFORMATION STORED IN THE FILE.

(vi) PROVIDE SCREEN SHOTS OF ALL THE RESULTS INCLUDING THE PROGRAM.

5. a) Write a Kernel module to display the scan code of the key being pressed and released using keyboard interrupt routine.

b) PROVIDE SCAN CODES FOR ALL THE KEYS ON THE KEYBOARD

c) PROVIDE A LOOKUP TABLE CONTAINING SCANCODE FOR EVERY KEY AND ITS CORRESPONDING KEY NAME

d) PROVIDE SCREEN SHOTS OF ALL THE RESULTS INCLUDING THE PROGRAM.

6. a) Write a Kernel module to identify a key being pressed or released on the keyboard and display the scan code and name of the key being pressed and released using keyboard interrupt routine.

This kernel module should detect all the keys on the keyboard (Upper case & Lower case Alphabets, Numerals 0-9, Special editing & arithmetic keys (; . , : ? > < / ' " [] { } \ | - = + _) (* & ^ % \$ # @ ! ~ space), Function keys F1 to F12, Ctrl(left & right), Backspace, Alt(left & right), insert, Delete, ins, del, Enter, shift (left & right), Home, End, NumLk, PageUp, Page Down, PgUp, PgDn, Esc, PrtSc, Pause and four arrow keys).

b) PROVIDE A LOOKUP TABLE CONTAINING SCANCODE FOR EVERY KEY, ITS CORRESPONDING KEY NAME AND ITS CORRESPONDING ASCII CODES

c) PROVIDE SCREEN SHOTS OF ALL THE RESULTS INCLUDING THE PROGRAM.

PART II : INTERPROCESS COMMUNICATION SOCKET PROGRAMMING

ALL MEMBERS INCLUDING THE TEAM LEADER MUST EXECUTE PART II QUESTIONS 9, 10, 11, 12 & 13 ON THEIR LAPTOPS/DESKTOPS AND PROVIDE THE SCREENSHOTS OF THE PROGRAMS & COMMANDS ENTERED ON THEIR SYSTEM, AND RESULTS OBTAINED ON THEIR SYSTEM.

HARDCOPY OF THE TEAM LEADER MUST CONTAIN ALL THESE SCREENSHOTS CLEARLY INDICATING THE NAME, ID OF THE MEMBERS INCLUDING THEIR SIGNATURE IN EVERY PAGE OF THE SCREEN SHOT SUBMITTED.

FOR QUESTIONS 11, 12 & 13, SERVER MUST BE SETUP ON TEAM LEADER'S MACHINE AND CLIENTS MUST BE SETUP ON DIFFERENT MEMBER MACHINES. FOR EXAMPLE, IF YOU HAVE SIX MEMBERS IN YOUR TEAM, THEN SERVER MUST BE SETUP ON TEAM LEADER'S MACHINE AND 5 CLIENTS MUST BE SETUP ON 5 MEMBER MACHINES.

YOU ARE REQUIRED TO CLEARLY INDICATE THE CORRECT ANSWERS FOR

ALL PARTS OF THE QUESTIONS a), b), c), e), f) UPTO u) IN ORDER TO GET FULL MARKS. ANY MISSING PART ANSWER WILL RESULT IN 0 MARKS

ANSWER FOR QUESTIONS 7 & 8 SHOULD BE TYPED BY THE INDIVIDUAL MEMBER IN HIS/HER REPORT

7. a) Describe all the steps required for developing Linux Kernel Modules.
b) Explain what is meant by socket programming.
c) Discuss in detail the stages involved in establishing connection between server and client(s) to exchange messages (Client-Server model).
d) Also provide State diagram for Server and Client model.
8. Describe in detail the following functions available in C to perform socket programming by clearly specifying the purpose and parameters required :
- a) socket b) setsockopt c) bind d) listen e) accept
f) connect g) read h) send i) select j) inet_pton k) htons
l) FD_ZERO m) FD_SET n) FD_CLR o) FD_ISSET
p) getpeername q) write r) htonl s) recv t)close u) inet_ntoa
9. Consider the following programs using sockets to communicate between the server and the client (i.e., to handle one client on a server)

Program Name : server.c

// Server side C program to demonstrate Socket programming

```
#include <stdio.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>
#include <netinet/in.h>
#include <string.h>
#define PORT 8080
```

```
int main(int argc, char const *argv[])
{
    int server_fd, new_socket, valread;
    struct sockaddr_in address;
    int opt = 1;
```



```

int addrlen = sizeof(address);
char buffer[1024] = {0}, schat[1024];
char *hello = "Hello from server";
char *bye = "bye";

printf("CREATING SERVER SOCKET .....\\n");
// Creating socket file descriptor
if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0)
{
    perror("socket creation failed");
    exit(EXIT_FAILURE);
}

printf("DEFINING SERVER SOCKET OPTIONS, FAMILY, ADDRESS &
PORT .....\\n");
// Forcefully attaching socket to the port 8080
if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR | SO_REUSEPORT, &opt,
sizeof(opt)))
{
    perror("setsockopt failed");
    exit(EXIT_FAILURE);
}
address.sin_family = AF_INET;
address.sin_addr.s_addr = INADDR_ANY;
address.sin_port = htons( PORT );

printf("BINDING SERVER SOCKET TO PORT 8080 .....\\n");
// Forcefully attaching socket to the port 8080
if (bind(server_fd, (struct sockaddr *)&address, sizeof(address))<0)
{
    perror("socket bind failed");
    exit(EXIT_FAILURE);
}

printf("SERVER LISTENING ON PORT 8080 FOR NEW CONNECTION.....\\n");
if (listen(server_fd, 3) < 0)
{
    perror("socket listen failed ");
    exit(EXIT_FAILURE);
}

if ((new_socket = accept(server_fd, (struct sockaddr *)&address,
(socklen_t*)&addrlen))<0)
{
    perror("socket accept failed");
    exit(EXIT_FAILURE);
}
printf("SERVER ACCEPTED NEW CONNECTION FROM A CLIENT ON PORT
8080 .....\\n");

valread = read( new_socket , buffer, 1024);

```

```

printf("%s\n",buffer );
send(new_socket , hello , strlen(hello) , 0 );
printf(" Ready for Chat....\n");
while(1)
{
    memset(buffer, 0, sizeof(buffer));
    valread = read( new_socket , buffer, 1024);
    printf("%s\n",buffer );
    memset(schat, 0, sizeof(schat));
    printf("server : ");
    fgets (schat, sizeof(schat), stdin);
    send(new_socket , schat , strlen(schat) , 0 );
    schat[strlen(schat)] = '\0';
    if(strncmp(schat , bye, strlen(bye))==0)
        break;
}
return 0;
}

```

Program Name : client.c

```

// Client side C program to demonstrate Socket programming
#include <stdio.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>
#include <netinet/in.h>
#include <string.h>
#include <arpa/inet.h>

#define PORT 8080

int main(int argc, char const *argv[])
{
    struct sockaddr_in address;
    int sock = 0, valread;
    struct sockaddr_in serv_addr;
    char *hello = "Hello from client";
    char buffer[1024] = {0}, cchat[1024];
    char *bye = "bye";

    printf("CREATING CLIENT SOCKET ..... \n");
    if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0)
    {
        printf("\n Socket creation error \n");
        return -1;
    }
}

```

```

printf("DEFINING CLIENT SOCKET FAMILY, ADDRESS & PORT .....\\n");
memset(&serv_addr, '0', sizeof(serv_addr));
serv_addr.sin_family = AF_INET;
serv_addr.sin_port = htons(PORT);
// Convert IPv4 and IPv6 addresses from text to binary form
if(inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr)<=0)
{
    printf("\\nInvalid address/ Address not supported \\n");
    return -1;
}

printf("CLIENT CONNECTING ON PORT 8080 TO COMMUNICATE WITH
SERVER.....\\n");
if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0)
{
    printf("\\nConnection Failed \\n");
    return -1;
}

send(sock , hello , strlen(hello) , 0 );
printf("Hello message sent\\n");
valread = read( sock , buffer, 1024);
printf("%s\\n",buffer );
printf(" Ready for Chat....\\n");
while(1)
{
    memset(buffer, 0, sizeof(buffer));
    memset(cchat, 0, sizeof(cchat));
    printf("CLIENT : ");
    fgets (cchat, sizeof(cchat), stdin);
    send(sock , cchat , strlen(cchat) , 0 );
    valread = read( sock , buffer, 1024);
    printf("%s\\n",buffer );
    cchat[strlen(cchat)] = '\\0';
    if(strncmp(cchat, bye, strlen(bye))==0)
        break;
}
return 0;
}

```

You are required to go through server.c and client.c programs and do the following:

- a) Compile the programs (using gcc -o server server.c && gcc -o client client.c).**
- b) Setup the server first by opening a new xterm and run the command ./server.**
- c) Setup the client next by opening a new xterm and run the command ./client.**

d) Now start a chat session by starting the chat from the client side (when you see the prompt CLIENT: enter your chat text and press enter to send it to server). You will see your chat text on server screen and the server will give you the prompt SERVER: to enter your chat text and sent it to the client. Reply to the client chat text by entering your reply chat text at the server prompt. In this way, you can continue your chat session chatting about your group members, about different subjects what you are learning now and other relevant topics of interest. **YOUR CHAT SESSION MUST ALSO CONTAIN YOUR ACTION PLAN FOR CARRYING OUT YOUR TERM PROJECT**

e) Provide all the chat sessions outputs of both server side and client side screens. Provide screen shots of all the results including the program.

f) Describe all the steps in setting up server and client by going through the programs.

10. Consider the program to get the IP address of the remote host by specifying the domain name

Program Name : gethostname.c

```
/* When connecting to a remote host , it is necessary to have its IP address.
*/
/* Function gethostbyname is used for this purpose.
*/
/* It takes the domain name as the parameter and returns a structure of type hostent.
*/
/* This structure has the ip information.
*/
/* It is present in netdb.h. Lets have a look at this structure
*/
/* Description of data base entry for a single host.
*/
/*      struct hostent
*/
/*      {
*/
/*          char *h_name;          Official name of host.
*/
/*          char **h_aliases;      Alias list.
*/
/*          int  h_addrtype;       Host address type.
```

```

/*      int  h_length;      Length of address.
*/
/*      char **h_addr_list;  List of addresses from name server.
*/
/*      };
*/
/*      The h_addr_list has the IP addresses.
*/
#include<stdio.h>           //printf
#include<string.h>          //strcpy
#include<sys/socket.h>
#include<netdb.h>           //hostent
#include<arpa/inet.h>

int main(int argc , char *argv[])
{
    char *hostname;
    char ip[100];
    struct hostent *he;
    struct in_addr **addr_list;
    int i;
    if(argc == 2 )
    {
        hostname = argv[1];
        if ( (he = gethostbyname( hostname ) ) == NULL)
        {
            //gethostbyname failed
            perror("gethostbyname");
            return 1;
        }
        //Cast the h_addr_list to in_addr , since h_addr_list also has the ip address in long format
        only
        addr_list = (struct in_addr **) he->h_addr_list;

        for(i = 0; addr_list[i] != NULL; i++)
        {
            //Return the first one;
            strcpy(ip , inet_ntoa(*addr_list[i]) );
            printf("ip = %s \n", ip);
        }

        printf(" %s resolved to : %s \n" , hostname , ip);
    }
    else
        printf("Usage : ./gethostname domain_name \n");
    return 0;
}

```

Compile using gcc -o gethostname gethostname.c

Execute using ./gethostname domain_name

For example :

`$/gethostname www.google.com`

- a) TRY OUT FOR ATLEAST 30 DIFFERENT DOMAIN NAMES**
- b) PROVIDE ALL THE SCREENSHOTS OF THE PROGRAM & COMMANDS ENTERED, AND RESULTS OBTAINED**

11. Consider the following Program Handling multiple clients on server without multi threading, but using Select() command

- A better way to handle multiple clients is by using select() linux command.
- Select command allows to monitor multiple file descriptors, waiting until one of the file descriptors become active.
- For example, if there is some data to be read on one of the sockets select will provide that information.
- Select works like an interrupt handler, which gets activated as soon as any file descriptor sends any data.
- Data structure used for select - fd_set
- It contains the list of file descriptors to monitor for some activity. There are four functions associated with fd_set: fd_set readfds;
- // Clear an fd_set
- FD_ZERO(&readfds);
- // Add a descriptor to an fd_set
- FD_SET(master_sock, &readfds);
- // Remove a descriptor from an fd_set
- FD_CLR(master_sock, &readfds);
- //If something happened on the master socket , then its an incoming connection
- FD_ISSET(master_sock, &readfds);
- Activating select:
- activity = select(max_fd + 1 , &readfds , NULL , NULL , NULL);

☐ **Handling multiple clients on server without multi threading**

//PROGRAM : A simple server side code, which echos back the received //message.

//filename : servernmt.c

//Handle multiple socket connections with select and fd_set on Linux

```
#include <stdio.h>
#include <string.h> //strlen
#include <stdlib.h>
#include <errno.h>
#include <unistd.h> //close
#include <arpa/inet.h> //close
```

```

#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <sys/time.h>          //FD_SET, FD_ISSET, FD_ZERO macros
#define TRUE  1
#define FALSE 0
#define PORT 8888

int main(int argc , char *argv[])
{
    int opt = TRUE;
    int master_socket , addrlen , new_socket , client_socket[30] , max_clients = 30 , activity, i ,
        valread , sd;
    int max_sd;
    struct sockaddr_in address;
    char buffer[1025]; //data buffer of 1K

    //set of socket descriptors
    fd_set readfds;
    //a message
    char *message = "ECHO Daemon v1.0 \r\n";

    //initialise all client_socket[] to 0 so not checked

    for (i = 0; i < max_clients; i++)
    {
        client_socket[i] = 0;
    }

    //create a master socket
    if( (master_socket = socket(AF_INET , SOCK_STREAM , 0)) == 0)
    {
        perror("socket creation failed");
        exit(EXIT_FAILURE);
    }

    //set master socket to allow multiple connections,
    //this is just a good habit, it will work without this
    if( setsockopt(master_socket, SOL_SOCKET, SO_REUSEADDR, (char *)&opt, sizeof(opt)) <
    0 )
    {
        perror("setsockopt");
        exit(EXIT_FAILURE);
    }

    //type of socket created
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons( PORT );

    //bind the socket to localhost port 8888

```

```

if (bind(master_socket, (struct sockaddr *)&address, sizeof(address))<0)
{
    perror("bind failed");
    exit(EXIT_FAILURE);
}
printf("Listener on port %d \n", PORT);
//try to specify maximum of 3 pending connections for the master socket
if (listen(master_socket, 3) < 0)
{
    perror("listen");
    exit(EXIT_FAILURE);
}
//accept the incoming connection
addrlen = sizeof(address);
puts("Waiting for connections ...");
while(TRUE)
{
    //clear the socket set
    FD_ZERO(&readfds);
    //add master socket to set
    FD_SET(master_socket, &readfds);
    max_sd = master_socket;
    //add child sockets to set
    for ( i = 0 ; i < max_clients ; i++)
    {
        //socket descriptor
        sd = client_socket[i];
        //if valid socket descriptor then add to read list
        if(sd > 0)
            FD_SET( sd , &readfds);
        //highest file descriptor number, need it for the select function
        if(sd > max_sd)
            max_sd = sd;
    }

    //wait for an activity on one of the sockets , timeout is NULL ,
    //so wait indefinitely
    activity = select( max_sd + 1 , &readfds , NULL , NULL , NULL);

    if ((activity < 0) && (errno!=EINTR))
    {
        printf("select error");
    }

    //If something happened on the master socket ,
    //then its an incoming connection
    if (FD_ISSET(master_socket, &readfds))
    {
        if ((new_socket = accept(master_socket, (struct sockaddr *)&address,
(socklen_t*)&addrlen))<0)
        {
            perror("accept");
        }
    }
}

```



```

        exit(EXIT_FAILURE);
    }

    //inform user of socket number - used in send and receive commands
    printf("New connection , socket fd is %d , ip is : %s , port : %d \n" , new_socket ,
inet_ntoa(address.sin_addr) , ntohs
        (address.sin_port));

    //send new connection greeting message
    if( send(new_socket, message, strlen(message), 0) != strlen(message) )
    {
        perror("send");
    }

    puts("Welcome message sent successfully");

    //add new socket to array of sockets
    for (i = 0; i < max_clients; i++)
    {
        //if position is empty
        if( client_socket[i] == 0 )
        {
            client_socket[i] = new_socket;
            printf("Adding to list of sockets as %d\n" , i);

            break;
        }
    }
}

//else its some IO operation on some other socket
for (i = 0; i < max_clients; i++)
{
    sd = client_socket[i];

    if (FD_ISSET( sd , &readfds))
    {
        //Check if it was for closing , and also read the
        //incoming message
        if ((valread = read( sd , buffer, 1024)) == 0)
        {
            //Somebody disconnected , get his details and print
            getpeername(sd , (struct sockaddr*)&address , (socklen_t*)&addrlen);
            printf("Host disconnected , ip %s , port %d \n" , inet_ntoa(address.sin_addr) ,
ntohs(address.sin_port));

            //Close the socket and mark as 0 in list for reuse
            close( sd );
            client_socket[i] = 0;
        }

        //Echo back the message that came in
        else

```

```

    {
        //set the string terminating NULL byte on the end
        //of the data read
        buffer[valread] = '\0';
        send(sd , buffer , strlen(buffer) , 0 );
    }
}
}
}

return 0;
}

```

a) Compile the file and run the server.

`$gcc -o servernmt servernmt.c`

b) Now setup the server on the machine by entering the command :

`./servernmt`

c) Use telnet to connect to the server as a client from other xterms.

Try running on different machines/xterms using following command: `telnet localhost 8888`

d) PROVIDE SCREEN SHOTS OF ALL THE RESULTS INCLUDING THE PROGRAM.

12. Consider the following Program Handling multiple clients on server with multi threading

Filename : servermt.c

```

//Handle multiple socket connections with threads
//To handle every connection we need a separate handling code to run along
//with the main server accepting connections.
//One way to achieve this is using threads.
//The main server program accepts a connection and creates a new thread to
//handle
//communication for the connection, and then the server goes back to
//accept more connections.
//On Linux threading can be done with the pthread (posix threads) library.
// use threads to create handlers for each connection the server accepts

```

```

#include<stdio.h>
#include<string.h> //strlen

```

```

#include<stdlib.h> //strlen
#include<sys/socket.h>
#include<arpa/inet.h> //inet_addr
#include<unistd.h> //write
#include<pthread.h> //for threading , link with lpthread
void *connection_handler(void *);

int main(int argc , char *argv[])
{
    int socket_desc , new_socket , c , *new_sock;
    struct sockaddr_in server , client;
    char *message;
//Create socket
    socket_desc = socket(AF_INET , SOCK_STREAM , 0);
    if (socket_desc == -1)
    {
        printf("Could not create socket");
    }

    //Prepare the sockaddr_in structure
    server.sin_family = AF_INET;
    server.sin_addr.s_addr = INADDR_ANY;
    server.sin_port = htons( 9999 );

    //Bind
    if( bind(socket_desc,(struct sockaddr *)&server , sizeof(server)) < 0)
    {
        puts("bind failed");
        return 1;
    }
    puts("bind done");

//Listen
    listen(socket_desc , 3);

    //Accept and incoming connection
    puts("Waiting for incoming connections...");
    c = sizeof(struct sockaddr_in);
    while( (new_socket = accept(socket_desc, (struct sockaddr *)&client, (socklen_t*)&c)) )
    {
        puts("Connection accepted");

        //Reply to the client
        message = "Hello Client , I have received your connection. And now I will assign a
handler for you\n";
        write(new_socket , message , strlen(message));

        pthread_t sniffer_thread;
        new_sock = (int*)malloc(sizeof(int));
        *new_sock = new_socket;

```

```

    if( pthread_create( &sniffer_thread , NULL , connection_handler , (void*) new_sock)
< 0)
    {
        perror("could not create thread");
        return 1;
    }

    //Now join the thread , so that we dont terminate before the thread
    //pthread_join( sniffer_thread , NULL);
    puts("Handler assigned");
}

if (new_socket<0)
{
    perror("accept failed");
    return 1;
}

return 0;
}
/* * This will handle connection for each client * */
void *connection_handler(void *socket_desc)
{
    //Get the socket descriptor
    int sock = *(int*)socket_desc;
    int read_size;
    char message[2000] , client_message[2000], schat[2000], *bye, s[10], chat[2000], s1[100];
    bye = "bye";
    //Send some messages to the client
    strcpy(message, "To ");
    sprintf(s, "%d", sock);
    strncat(message, s, strlen(s));
    strcpy(s1, " : Greetings! I am your connection handler\n");
    strncat(message, s1, strlen(s1));
    write(sock , message , strlen(message));
    memset(message, 0, sizeof(message));
    read_size = read(sock , client_message , strlen(client_message));
    printf("From Client [%d] : %s \n", sock, client_message );
    strcpy(message, "To ");
    sprintf(s, "%d", sock);
    strncat(message, s, strlen(s));
    strcpy(s1, "Now we shall have a chat session : so type something \n");
    strncat(message, s1, strlen(s1));
    write(sock , message , strlen(message));
    //Receive a message from client
    while(1)
    {
        memset(client_message, 0, sizeof(client_message));
        read_size = read(sock , client_message , strlen(client_message));
        printf("From Client [%d] : %s\n", sock, client_message );
        memset(schat, 0, sizeof(schat));
        printf("server : ");
    }
}

```

```

fgets (schat, sizeof(schat), stdin);
strcpy(chat, "To ");
sprintf(s, "%d", sock);
strncat(chat, s, strlen(s));
strncat(chat, schat, strlen(schat));
send(sock, chat , strlen(chat) , 0 );
schat[strlen(schat)] = '\0';
if(strncmp(schat , bye, strlen(bye))==0)
    break;
}
puts("Client disconnected");
fflush(stdout);
//Free the socket pointer
free(socket_desc);
return 0;
}

```

a) Compile the file and run the server.

```

$gcc -o servernmt servermt.c
$./servermt

```

b) Use telnet to connect to the server as a client from other xterms.

Try running on different machines/xterms using following command: **telnet localhost 9999**

c) **PROVIDE SCREEN SHOTS OF ALL THE RESULTS INCLUDING THE PROGRAM.**

13. Write a program using sockets to handle multiple clients on a server (i.e., multiple clients should connect to the server and communicate by exchanging the messages).

a) You could have a multi-client chat session with the server - each client connecting to the same server and having a chat session with the same server.

b) **Provide the Screenshots of all the CHAT sessions of CLIENTS on different machines connected to the MAIN SERVER machine.**

c) **PROVIDE SCREEN SHOTS OF ALL THE RESULTS INCLUDING THE PROGRAM.**
